中国科学院计算技术研究所
INSTITUTE OF COMPUTING TECHNOLOGY, CHINESE ACADEMY OF SCIENCES

北京开源芯片研究院
BEIJING INSTITUTE OF OPEN SOURCE CHIP

香山开源处理器社区
XiangShan Open-Source Processor Community

XSAI (ξ)
From XiangShan to an AI-Ready CPU

The XiangShan Team

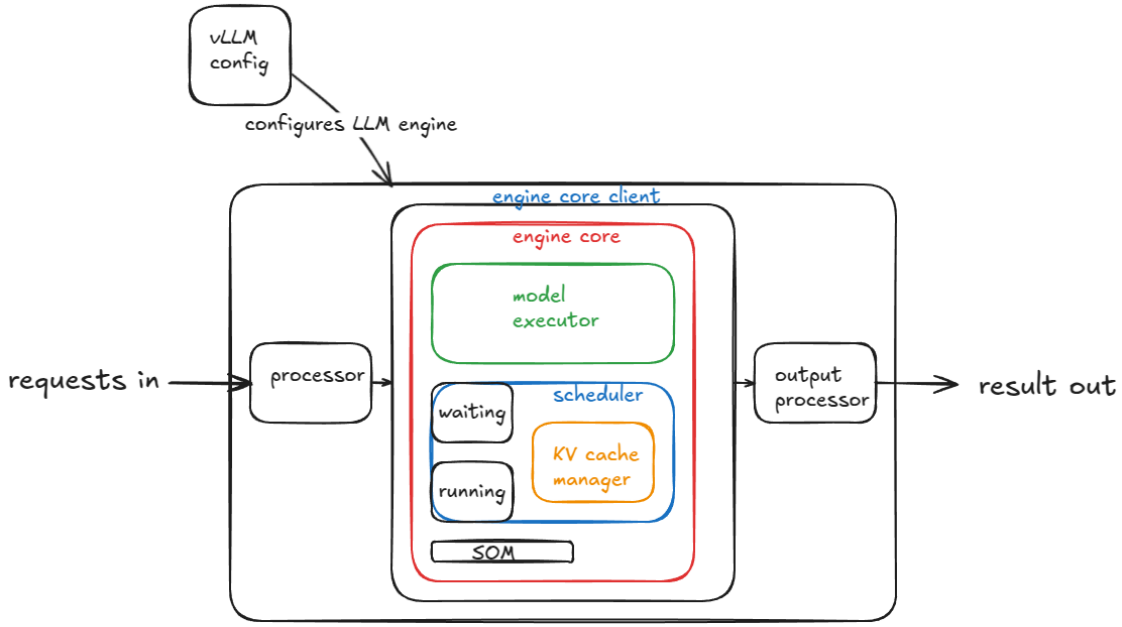
Agenda

- XSAI Design Philosophy
- Application Scenarios
- Technical Highlights
- Roadmap

AI is no longer just GEMM

AI inference workloads are becoming **increasingly complex**

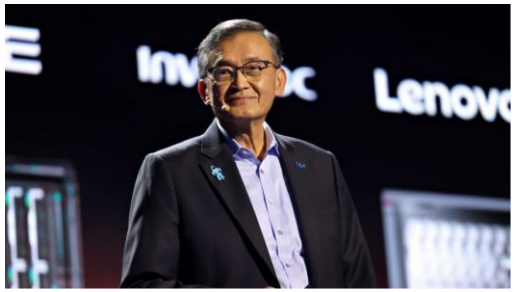
- Request scheduling
- Sampling
- Tool call
- Speculative decoding
  - Acceptance and rollback
- KV Cache
  - Allocation, sharing, selection, release, and offload



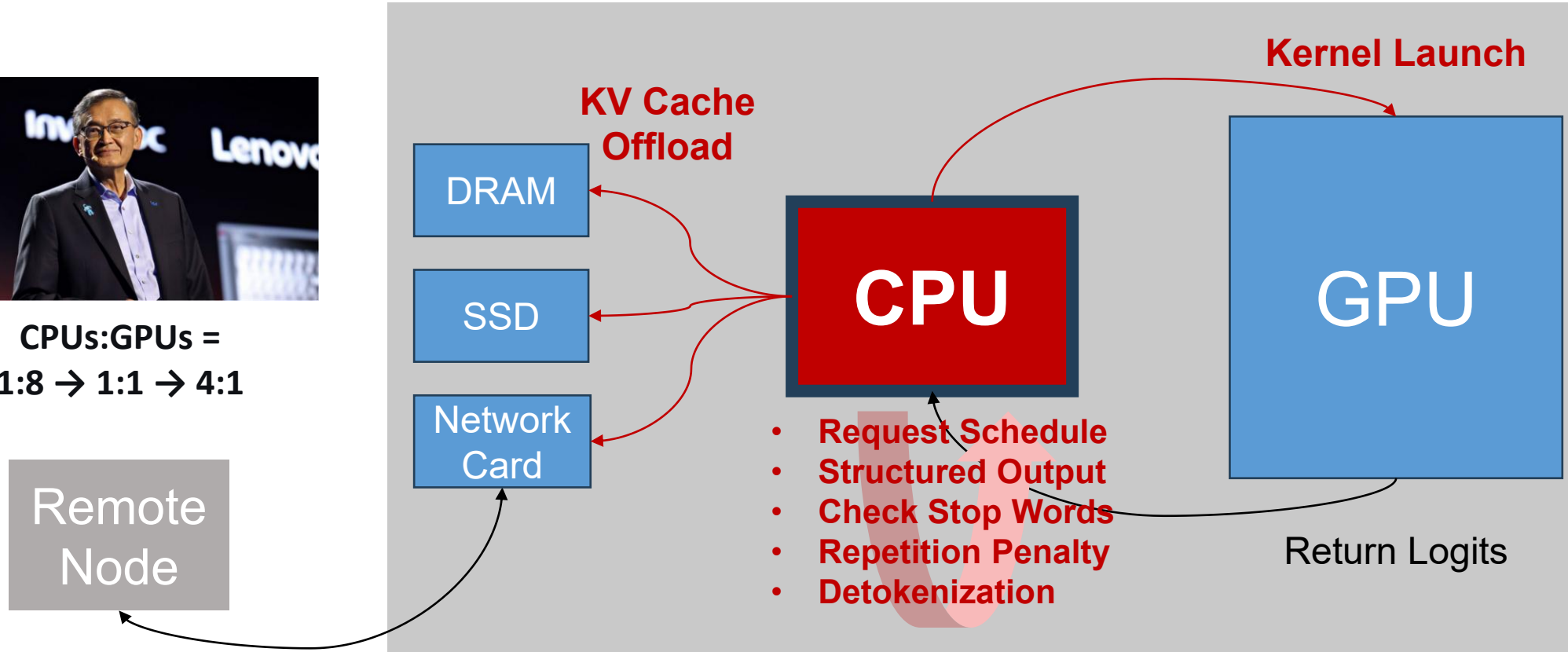
vLLM Components

From Host + Device to AI CPU

- AI workloads are becoming system workloads
- **Host CPU** is under **increasing pressure**



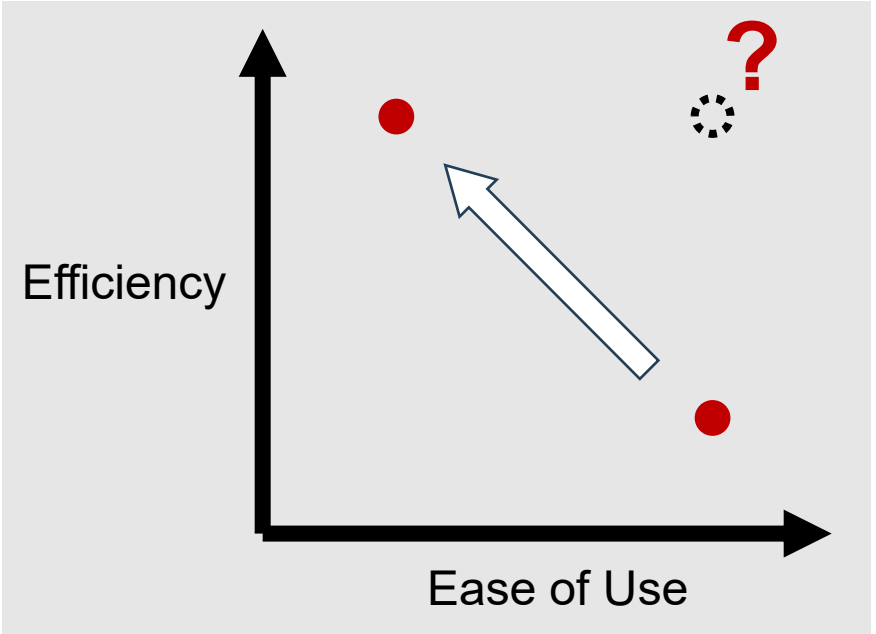
CPUs:GPUs =
1:8 → 1:1 → 4:1



From Host + Device to AI CPU

- Use **persistent kernel** to reduce CPU pressure
  - Blink
    - CPU-Free implementation for Continuous Batching and PagedAttention
  - Megakernels
    - A persistent kernel fuses the entire LLM inference process

- **Control** was handed over to the **GPU**
- **Complexity** is absorbed by **software**
  - Efficiency ↗↗↗
  - Modularity and maintainability ↘↘↘



AI CPU: LLM's native computing architecture

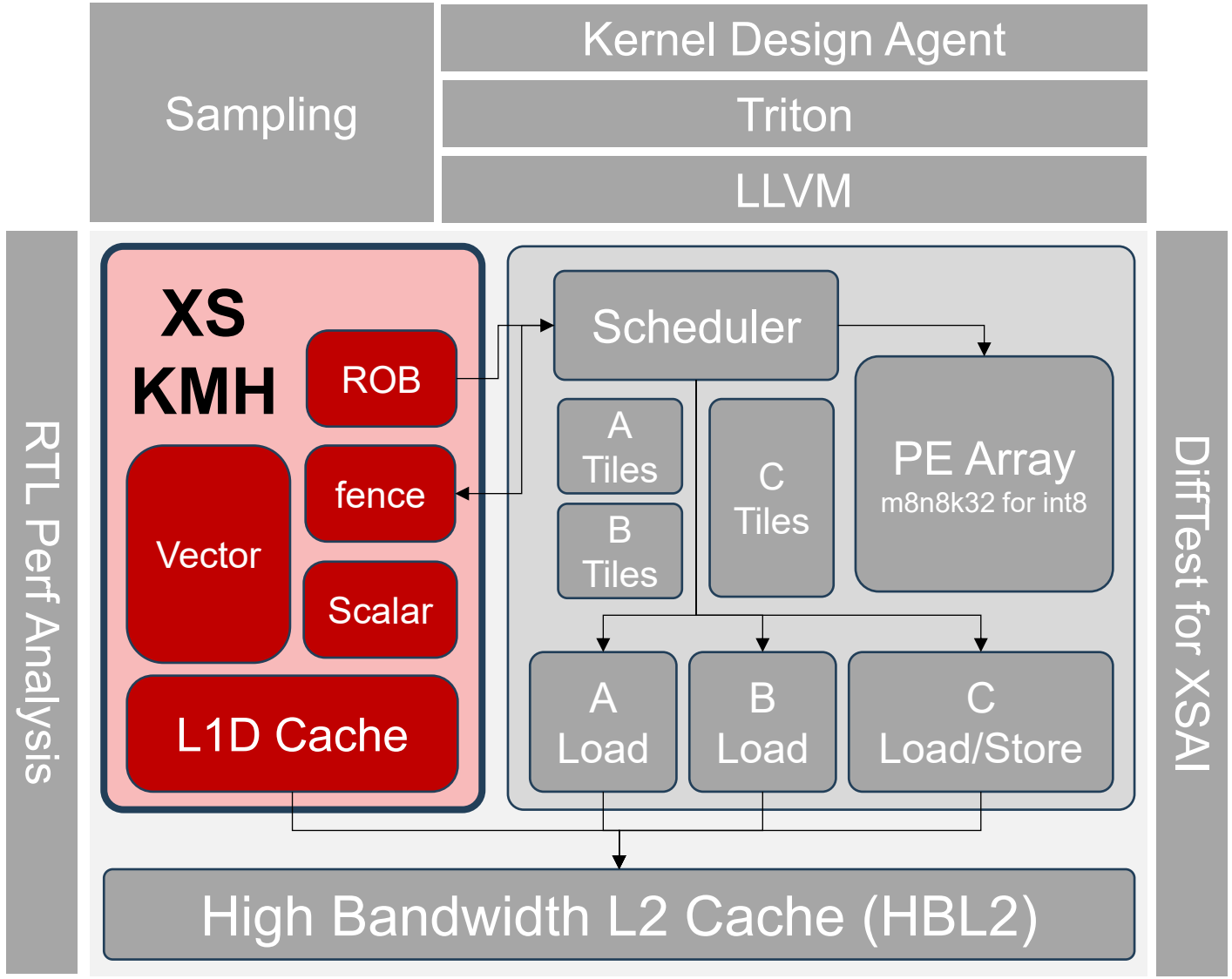
- **CPU-native** programming model
  - Reduce programming complexity
- **Tile-based** large matrix register file
  - Friendly to tile-based languages
  - Increase the granularity of a single instruction
- **Asynchronous** matrix instructions
  - Matrix and vector can overlap
- **High-performance general-purpose** compute
  - Efficiently handle irregular control flow
- **AI-oriented** memory hierarchy
  - High bandwidth and acceptable latency



Build AI CPU based on XiangShan

Why based on **Xiangshan**?

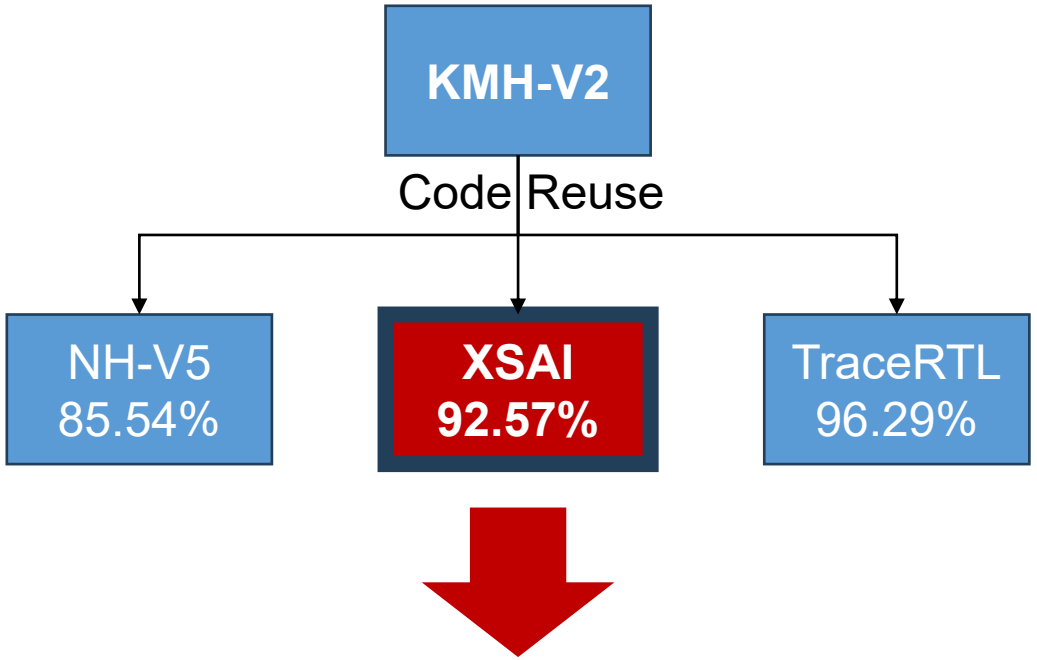
- **Open-source**
- **High performance**
- **Agility**



Build AI CPU based on XiangShan

- XiangShan: Open-source
- **XSAI is also open source**
  - Based on open-source KMH-V2 design
  - **Reuse 92.57% of the code**
  - Reduce human resource needs

https://github.com/OpenXiangShan/XSAI

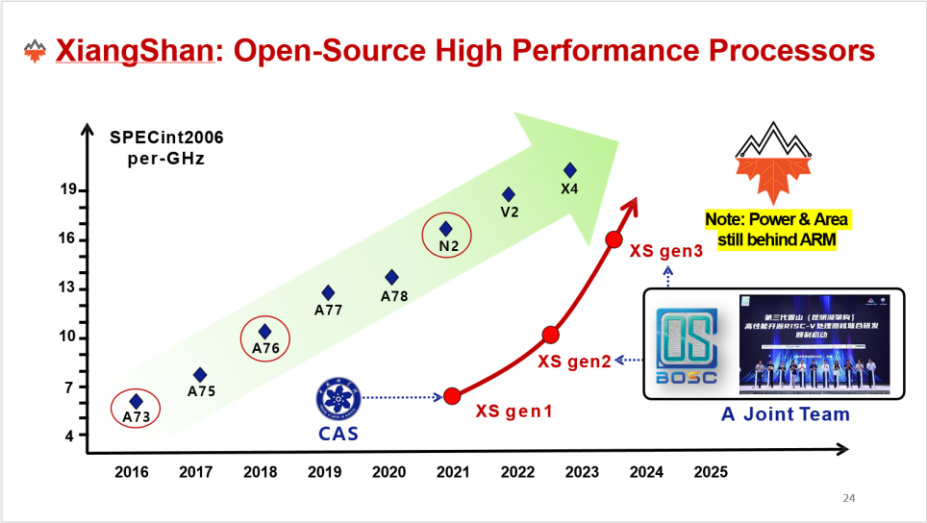


KMH lines of code and XSAI code reuse

	XiangShan	YunSuan	Utility	CoupledL2	huancun	OpenLLC	Other
KMH V2R2	134K	42K	7K	21K	12K	4K	
XSAI V2R2A	**94.57%**	**100%**	**100%**	**94.83%**	**100%**	**100%**	17K

Build AI CPU based on XiangShan

- XiangShan: High performance



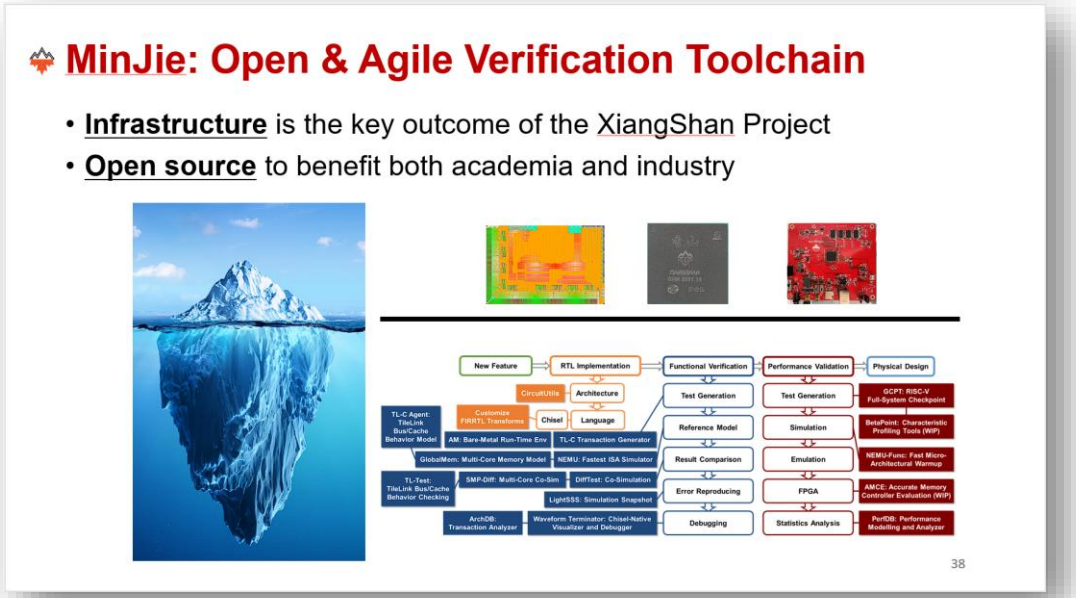
SPECint 2006 @ 3GHz	V2R2A	V2R2	SPECfp 2006 @ 3GHz	V2R2A	V2R2
400.perlbench	36.03	36.18	410.bwaves	67.35	66.73
401.bzip2	25.75	25.46	416.gamess	40.61	40.99
403.gcc	48.15	48.00	433.milc	44.38	45.12
429.mcf	63.26	60.63	434.zeusmp	51.65	51.61
445.gobmk	30.30	30.32	435.gromacs	33.50	33.60
456.hmmer	41.35	41.62	436.cactusADM	46.06	46.19
458.sjeng	30.25	30.24	437.leslie3d	48.31	47.97
462.libquantum	126.54	122.43	444.namd	28.82	28.86
464.h264ref	56.49	56.58	447.dealII	73.37	73.55
471.omnetpp	42.32	41.77	450.soplex	52.85	52.5
473.astar	29.23	29.19	453.povray	53.05	53.46
483.xalancbmk	71.39	72.84	454.Calculix	16.35	16.37
GEOMEAN	**44.92**	**44.66**	459.GemsFDTD	38.31	38.60
			465.tonto	36.65	36.66
			470.lbm	91.30	91.94
			481.wrf	40.25	40.70
			482.sphinx3	48.88	49.13
			GEOMEAN	**44.72**	**44.85**

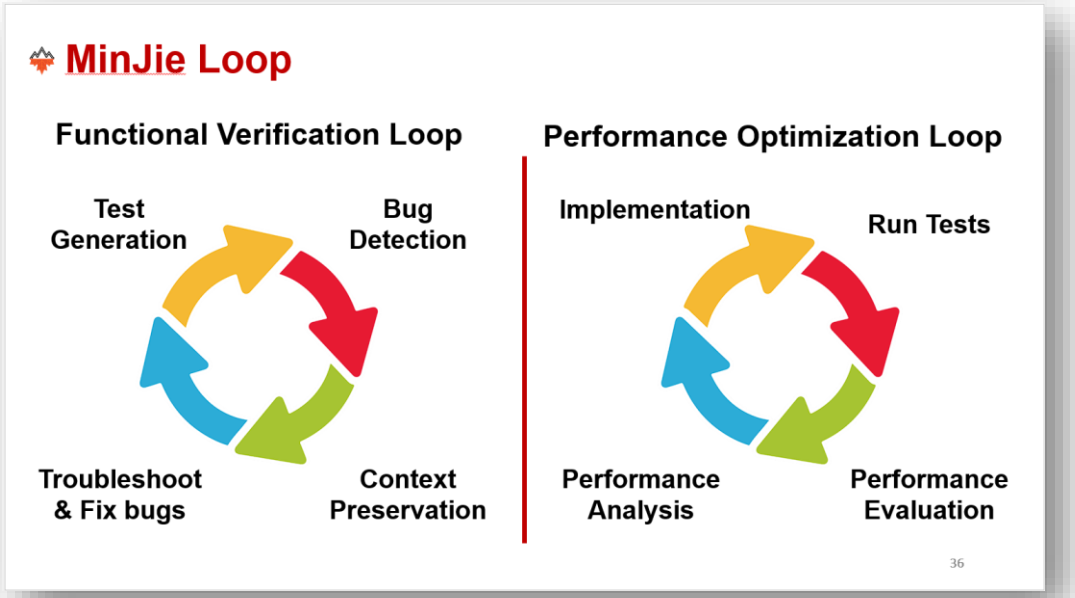
- **XSAI: Retain the performance of KMH V2R2**
  - SPECint 2006/GHz: 14.887->**14.976** (+0.58%)
  - SPECfp 2006/GHz: 14.950->**14.905** (-0.29%)

*XiangShan biweekly 100

Build AI CPU based on XiangShan

- XiangShan: Agility
  - MinJie open & agile verification toolchain



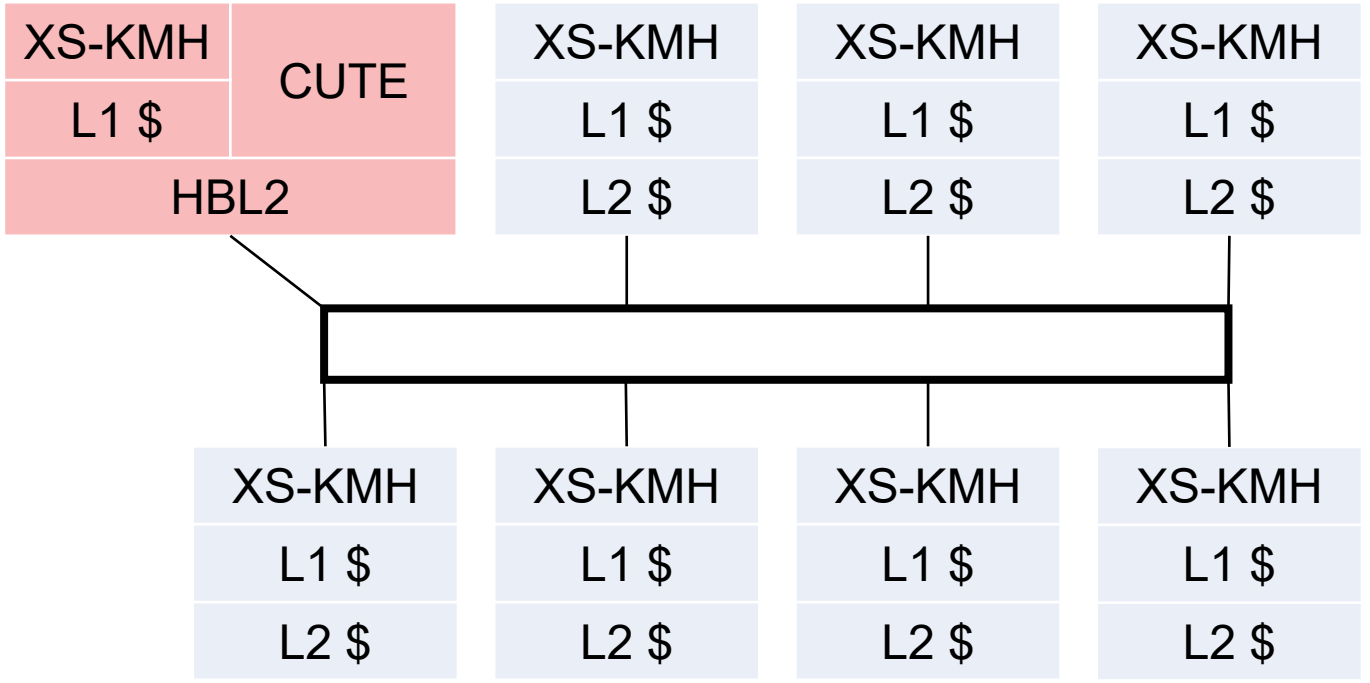


- **XSAI: Inheriting XiangShan's infrastructure**
  - **DiffTest, ChiselDB, NEMU, Simpoint, ……**

XSAI Application Scenarios

Consumer AIPC

- Qwen3.6-35B-A3B
  - MoE
  - MTP
- 2-channel DDR5
- 7 XS-KMH + 1 XSAI
- 1 user, 40 tokens/s

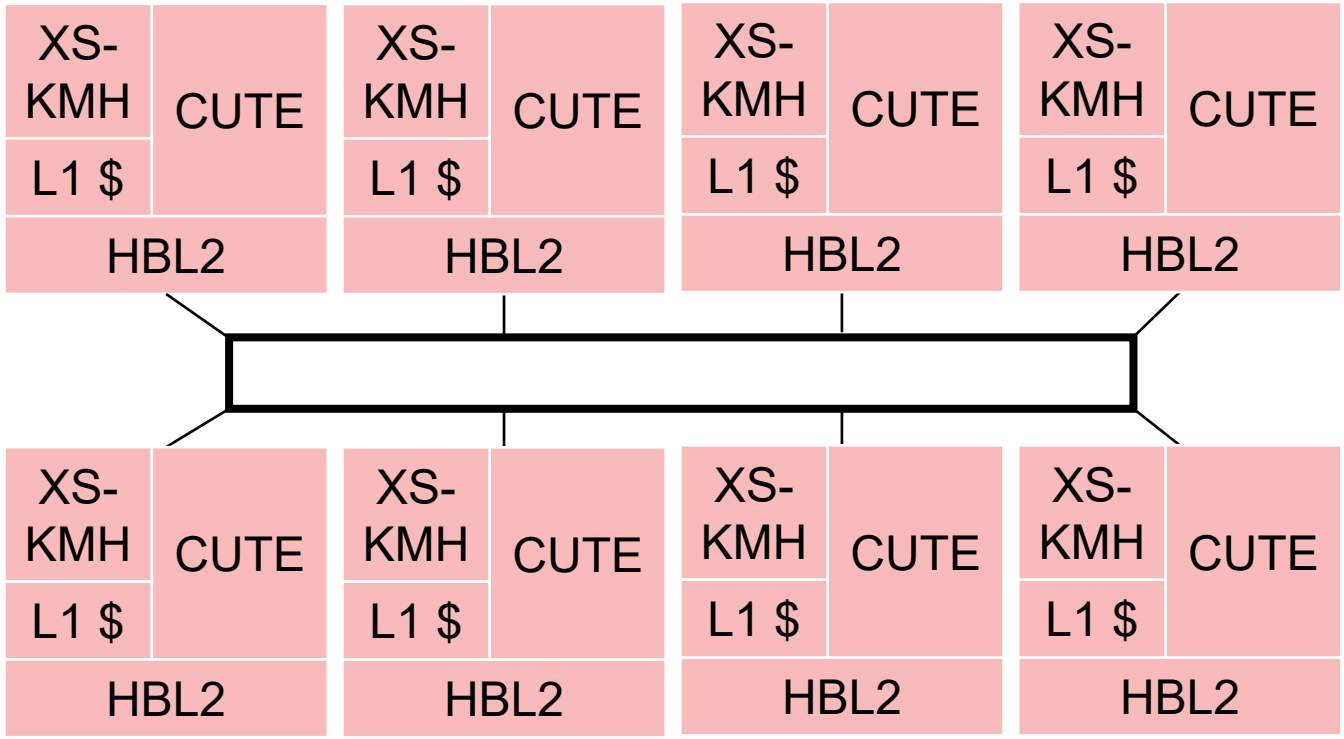


XSAI Application Scenarios

Chatbot Server

- DeepSeek-V4-Flash
  - MoE
  - MTP
  - Sparse Attention
- 8-channel DDR5
- 8 XSAI

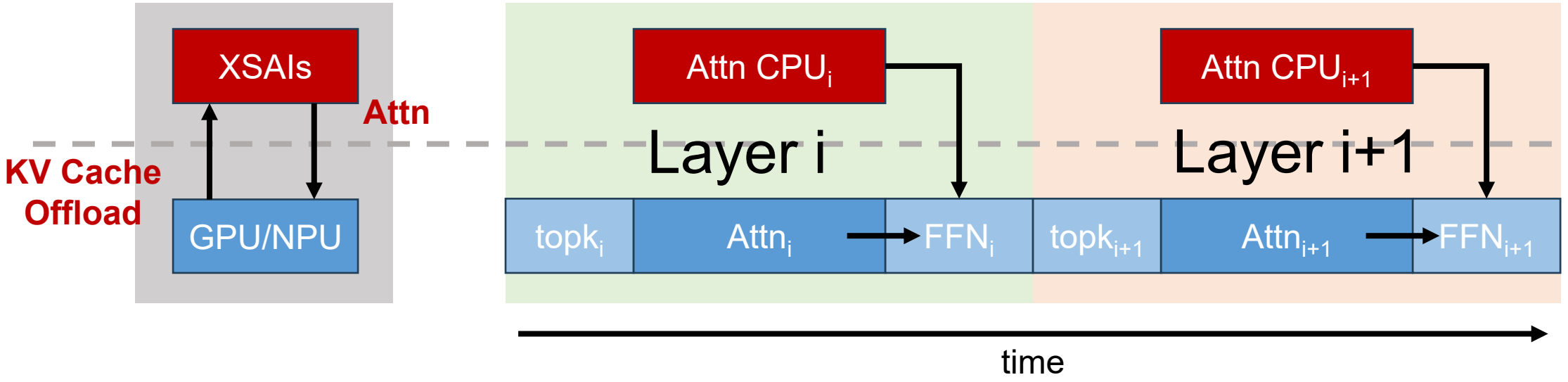
- 4 users, 20 tokens/s/user



XSAI Application Scenarios

Flagship Model Provider

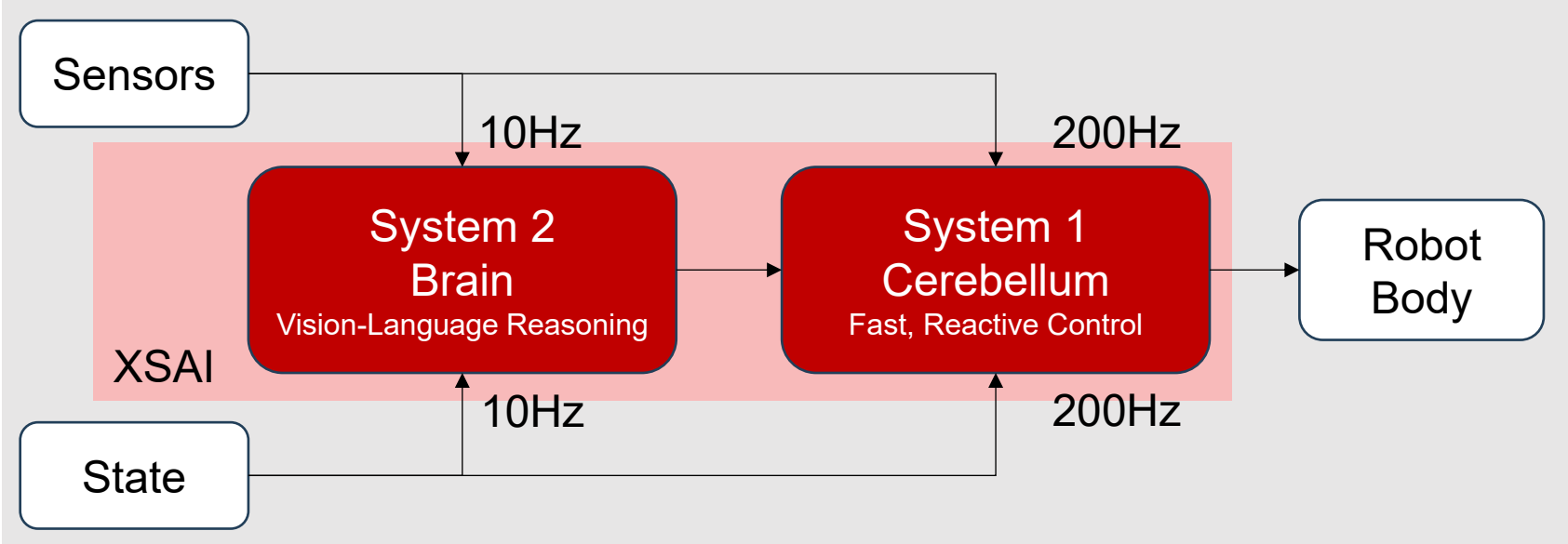
- DeepSeek-V4-Pro
- KV cache offloading in sparse attention
  - KV cache is offloaded from GPU/NPU to XSAIs' memory
  - HGCA, ScoutAttention: **AI CPU assist GPU/NPU in computing attention**



XSAI Application Scenarios

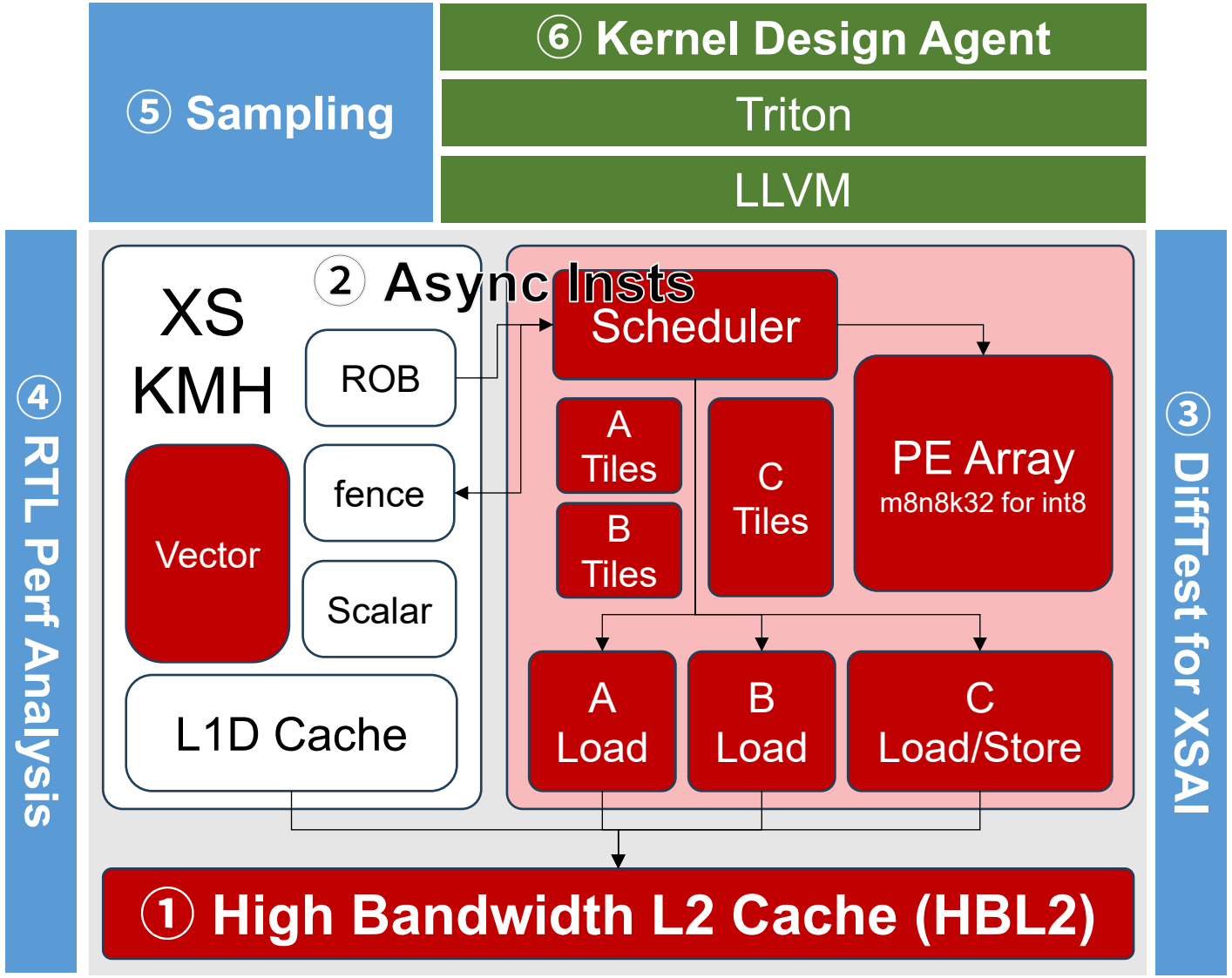
Embodied AI

- Simpler programming model
- Easier to write low-latency Megakernels
- 4-channel LPDDR5X



XSAI Technical Highlights

- AI CPU based on KMH
  1. High-bandwidth L2 cache
  2. Async matrix instructions
  3. Differential Testing
  4. Perf Analysis Toolkit
  5. Sampling
  6. Kernel Design Agent



High-Bandwidth L2 Cache

- Demands of matrix unit conflicts with high-perf CPU

	Primary demand	**what can be traded**	**Best buffer in existing memory hierarchy**
CPU	latency	capacity	L1 Cache
Matrix unit	capacity and hit bandwidth	latency	**?**

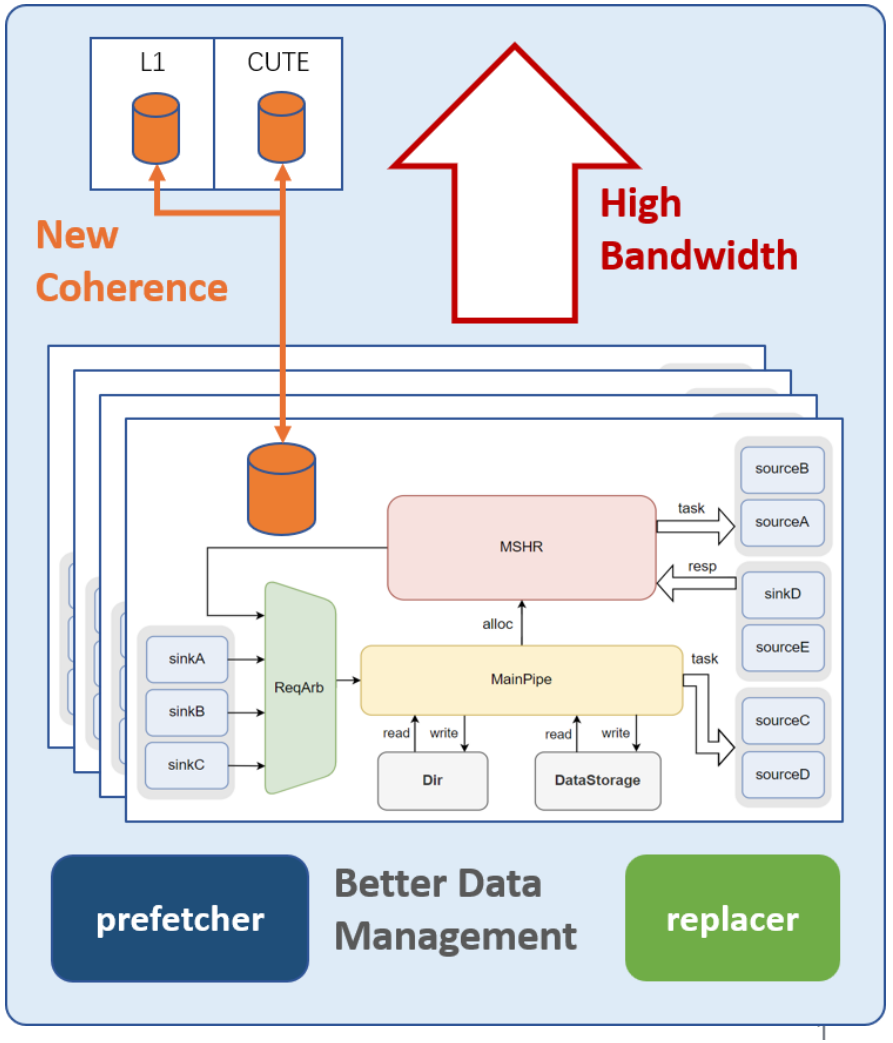
High-Bandwidth L2 Cache

- Demands of matrix unit conflicts with high-perf CPU

	Primary demand	what can be traded	Best buffer in existing memory hierarchy
CPU	latency	capacity	L1 Cache
Matrix unit	capacity and hit bandwidth	latency	**L2 Cache**

- High-bandwidth L2 cache (HBL2)
  - Capacity: 1-2 MB
  - Bandwidth: 256-512 Bytes per cycle
  - Latency: ~12 cycle hit latency

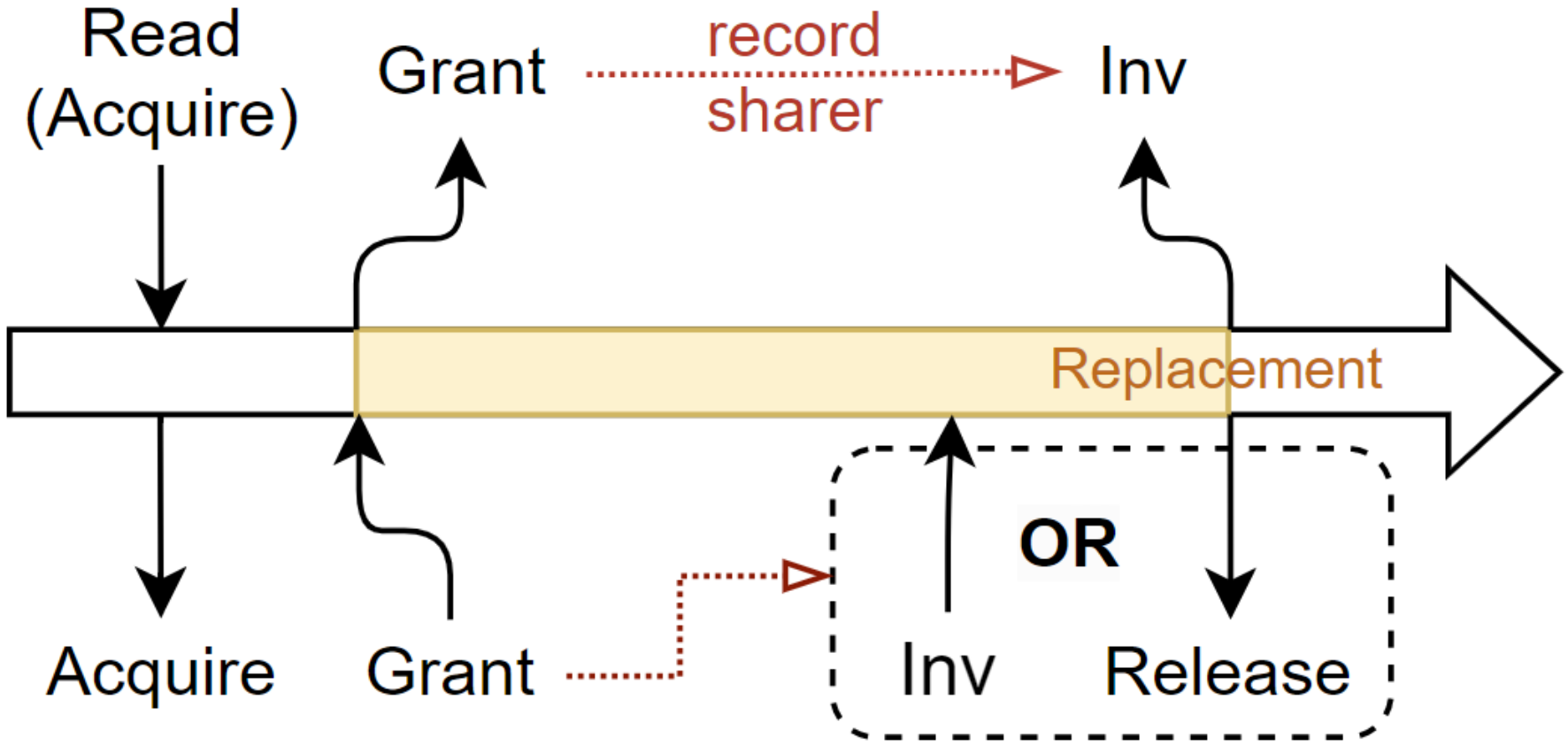
Effective hit bandwidth (Bytes/cycle)	HBL2 (8-bank)	Xiangshan L2 cache (4-bank)	XuanTie C910 L2 cache
	480	17	5.5



Source: https://chipsandcheese.com/p/alibaba-heads-xuantie-c910

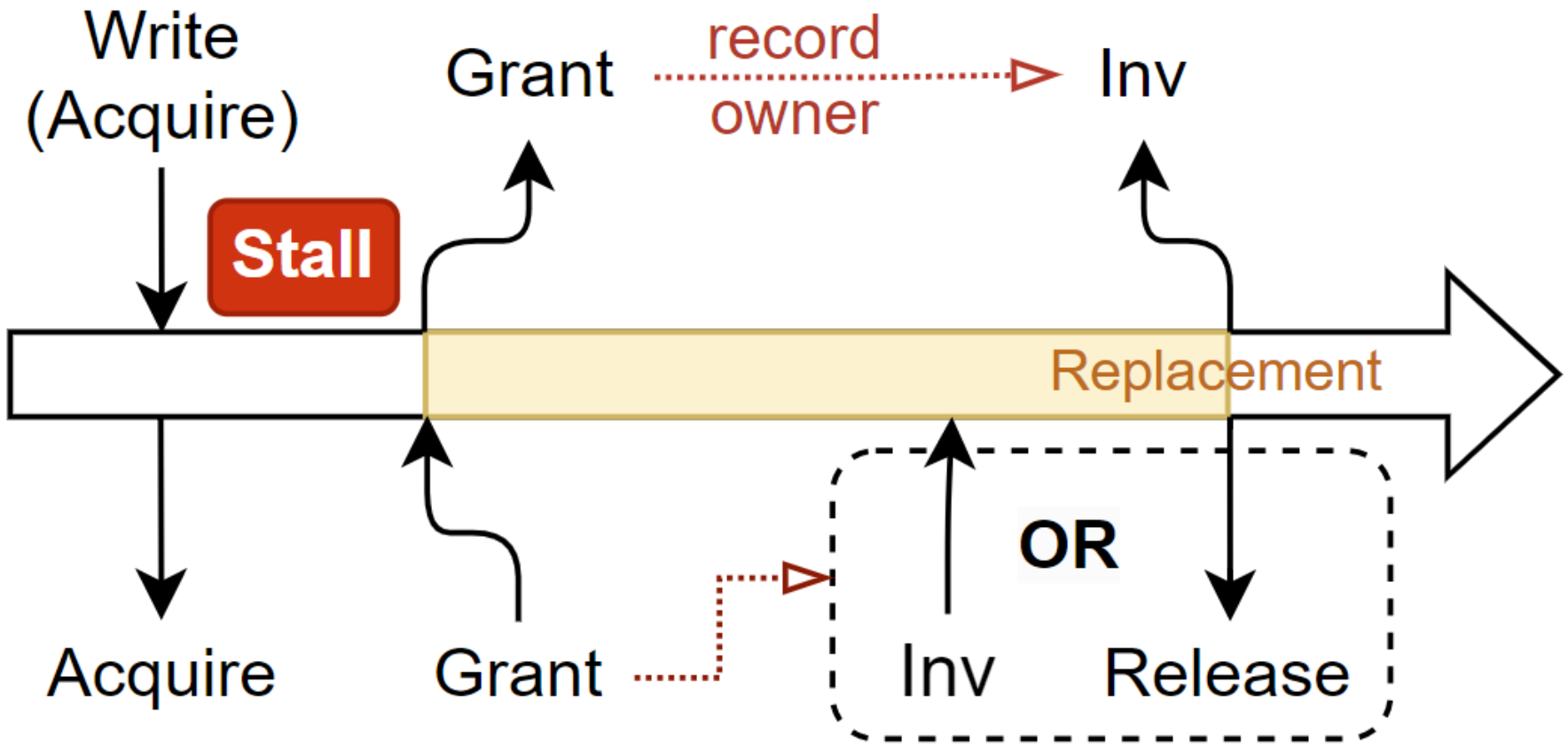
High-Bandwidth L2 Cache

- Cache coherence overhead (Read)



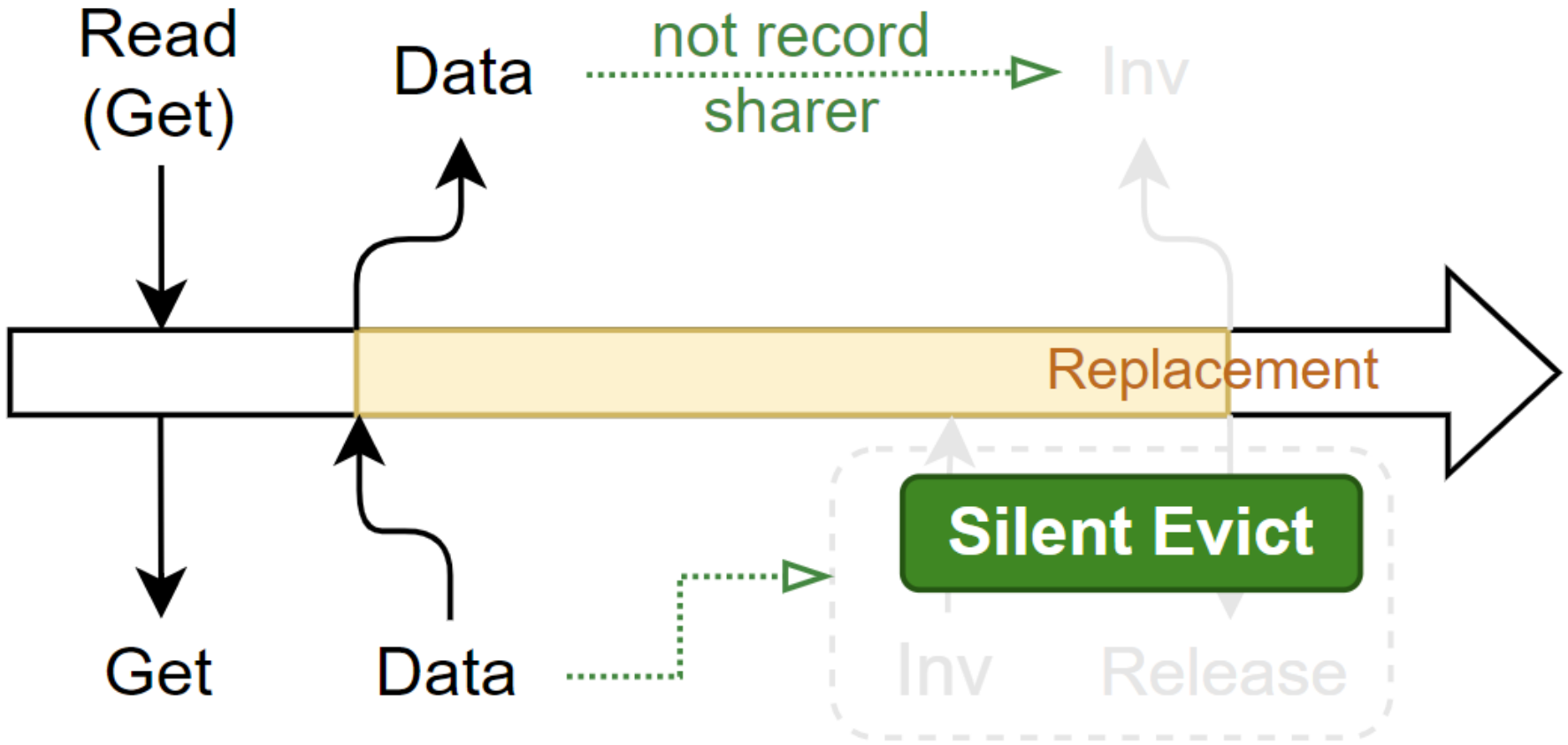
High-Bandwidth L2 Cache

- Cache coherence overhead (Write)



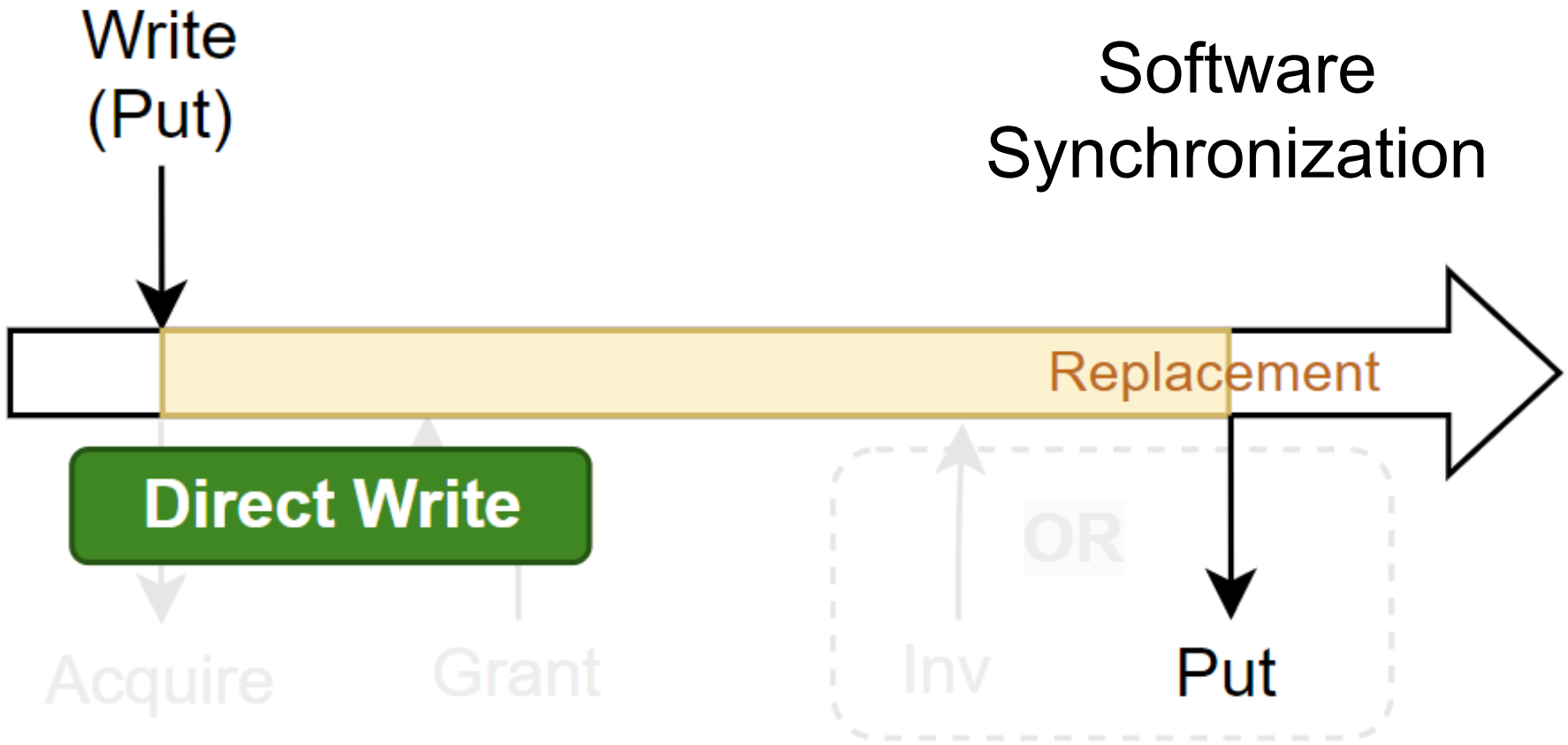
High-Bandwidth L2 Cache

- New cache coherence (Read)



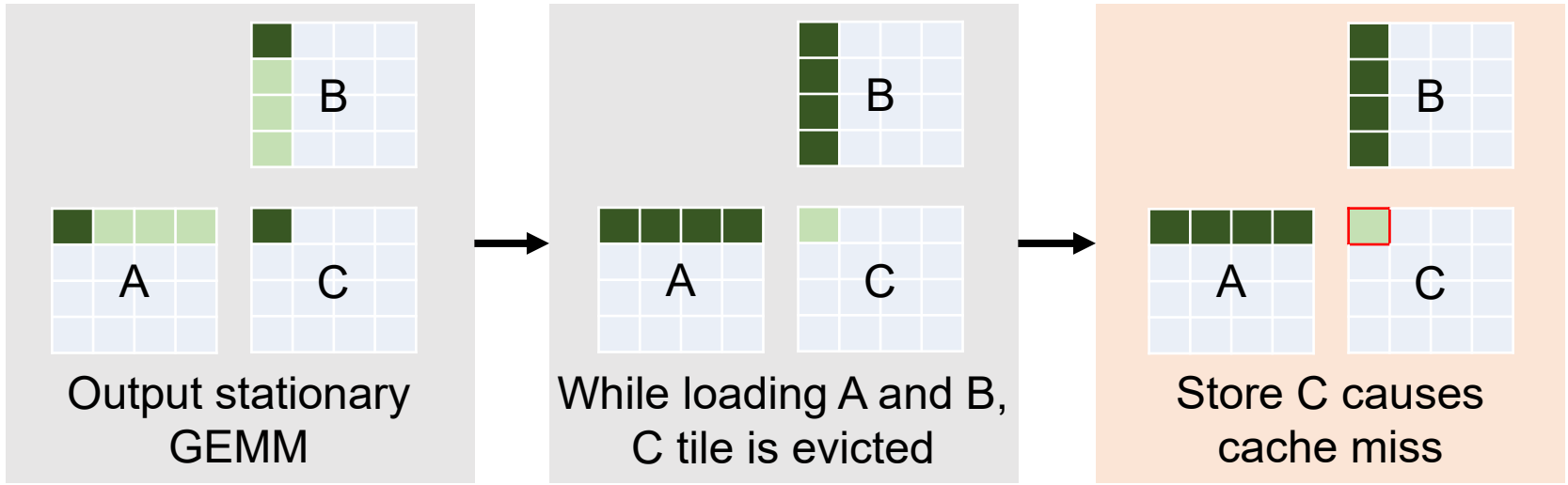
High-Bandwidth L2 Cache

- New cache coherence (Write)



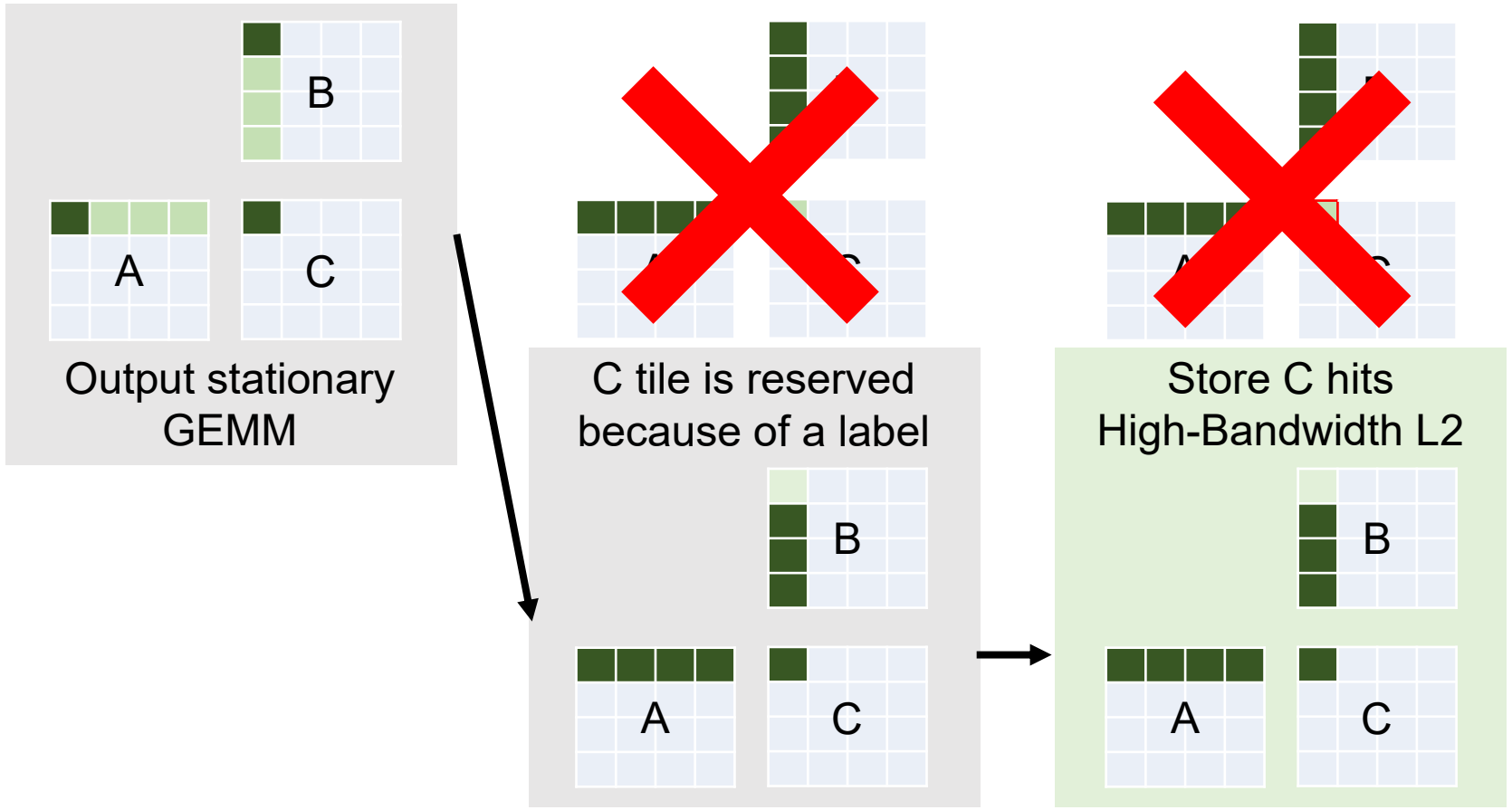
High-Bandwidth L2 Cache

- New data replacement
  - Example: Matrix C in Output stationary GEMM



High-Bandwidth L2 Cache

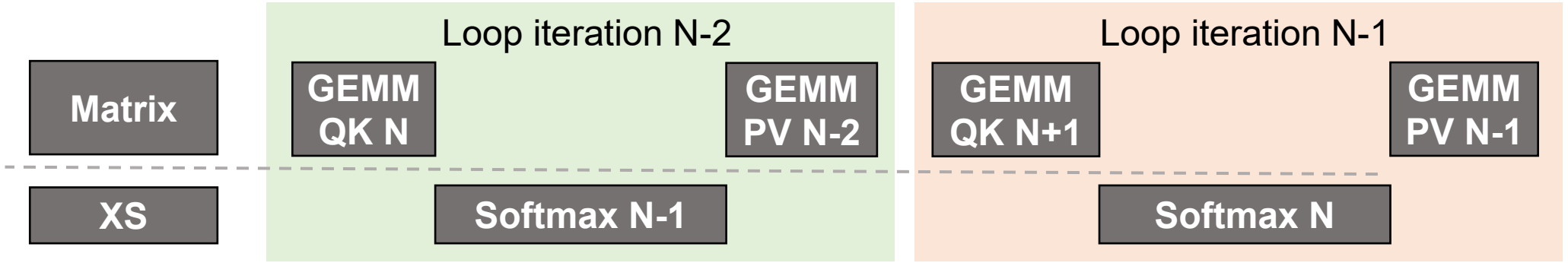
- New data replacement
  - Example: Matrix C in Output stationary GEMM



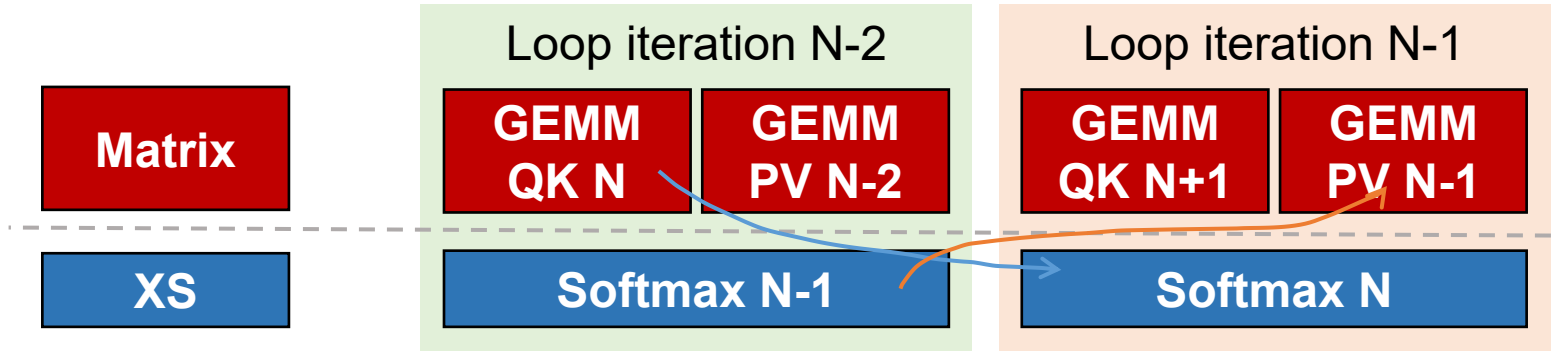
Improve Hit Rate

Asynchronous Matrix Instructions

- **Async mma/mld/mst** commit in scalar core without execution
  - Precise exception: VA-PA translation & permission check in scalar core



w/o Async Matrix: GEMM and softmax **cannot overlap**

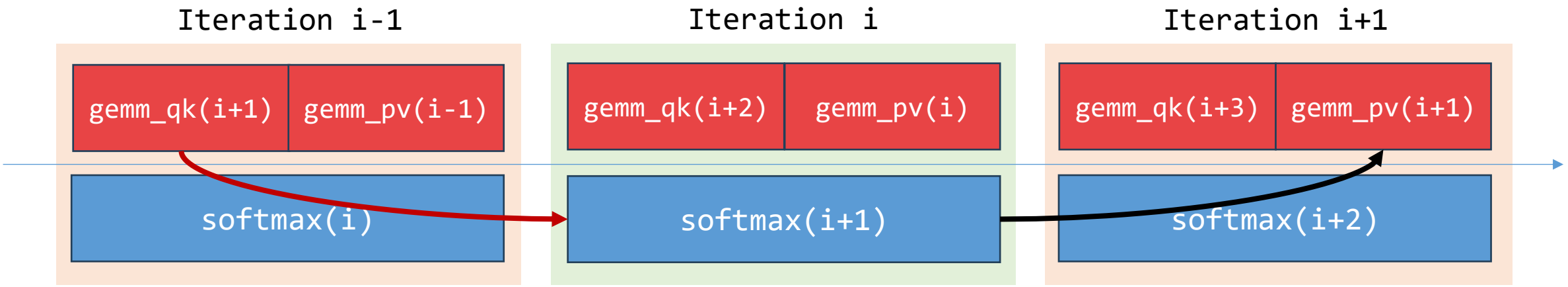


w/ Async Matrix: **3-stage FA3 overlap GEMM with softmax**

Asynchronous Matrix Instructions

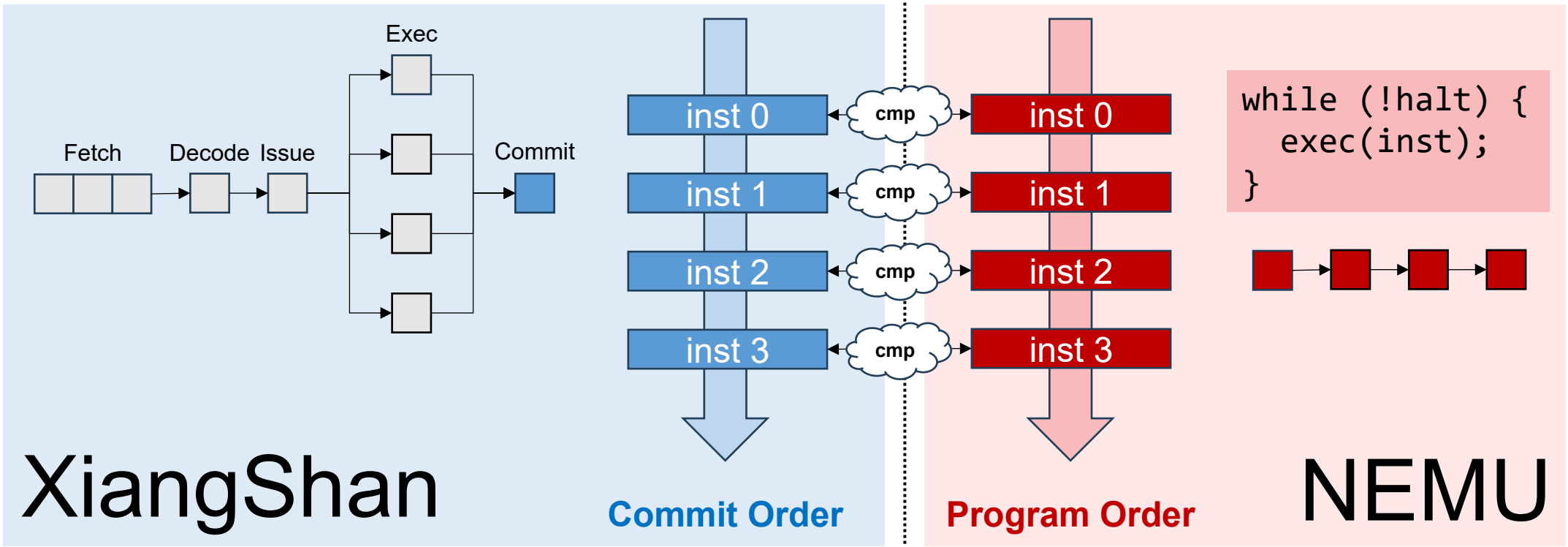
- Release-Acquire style sync
  - Matrix stores are invisible to scalar/vector
  - release/acquire to enforce dependency
- **Matrix and vector can overlap**

```
// main loop
for (int i = 0; i < n - 2; ++i) {
  gemm_qk(i + 2);
  mrelease(tok);
  gemm_pv(i);
  macquire(tok, i + 2);
  softmax(i + 1);
}
```



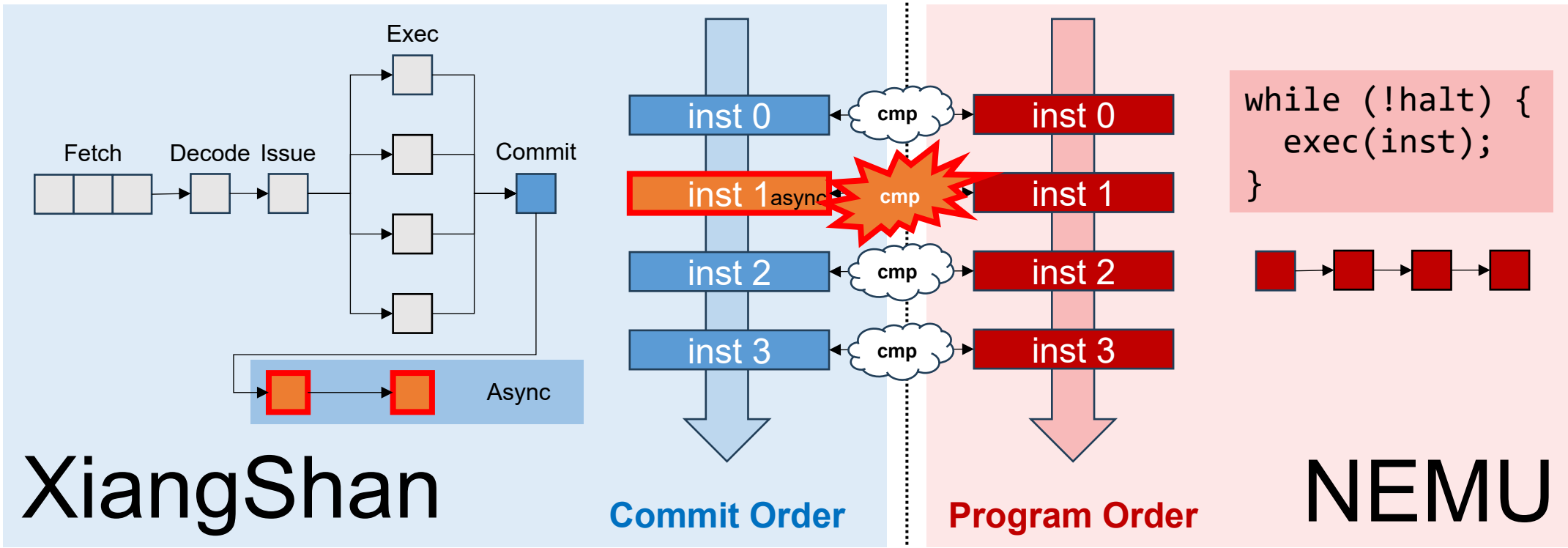
Traditional Differential Testing

- Traditional DiffTest in XiangShan requires **an order** for comparison
  - **Compare them one by one in program/commit order**
  - Difference will be reported



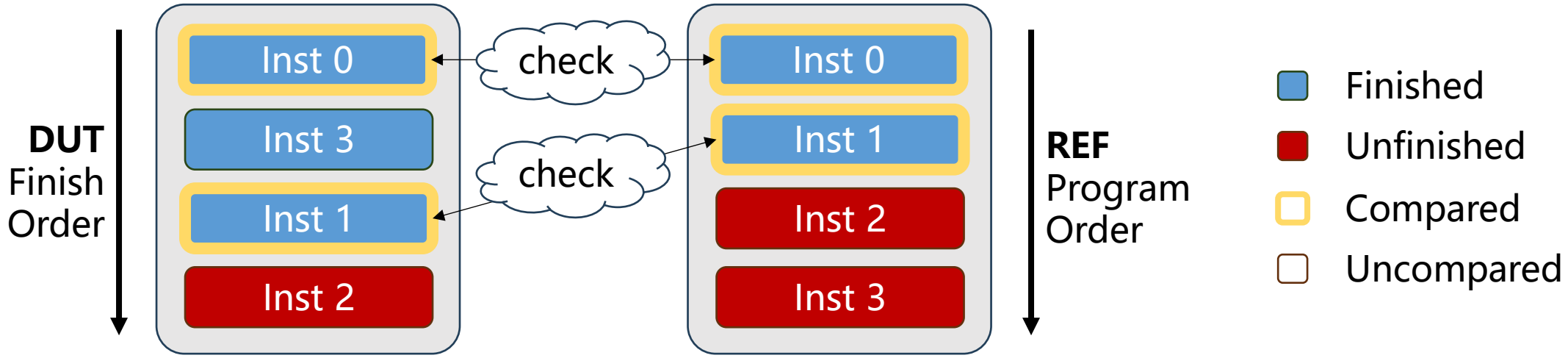
Differential Testing for Async Instructions

- Async instructions **violate the order**
  - Have not yet been executed at commit stage
  - **Unable to be compared during commit**



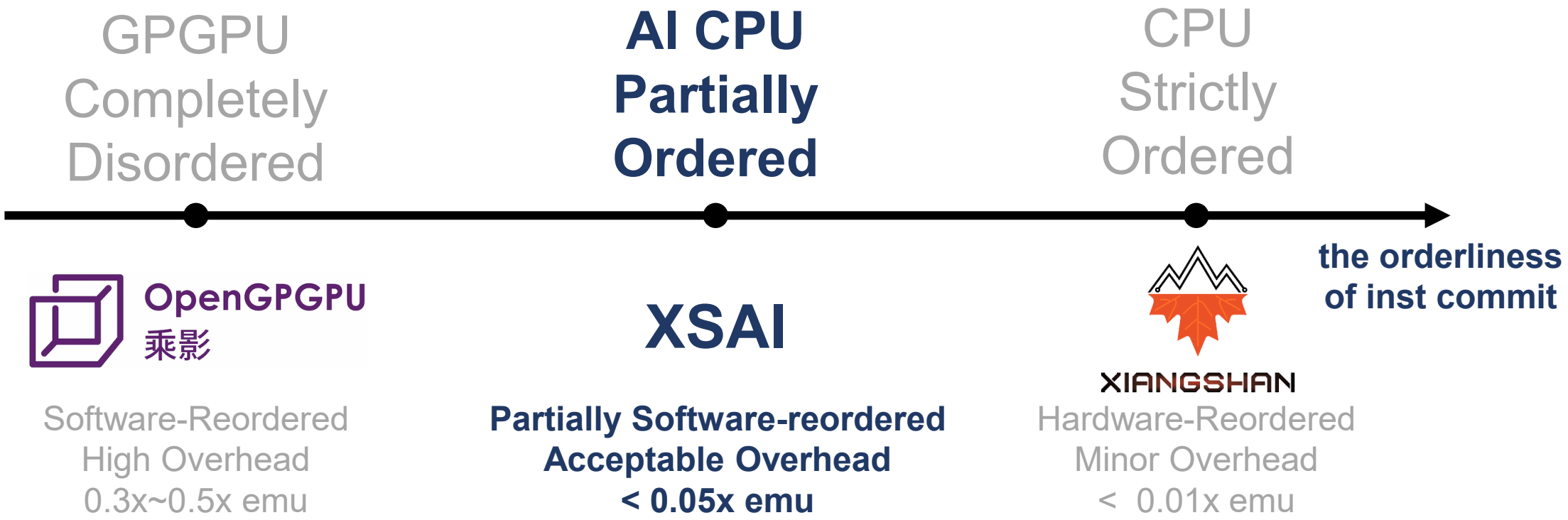
Differential Testing for Async Instructions

- Ventus (OpenGPGPU): Use a **software ROB** to do DiffTest
  - Reorder instructions by software
  - Compare a finished prefix of insts
  - Difference will be reported

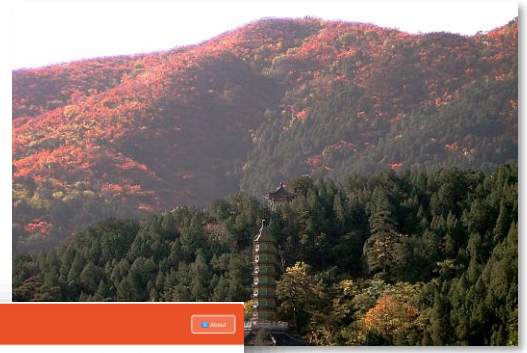


Differential Testing for XSAI

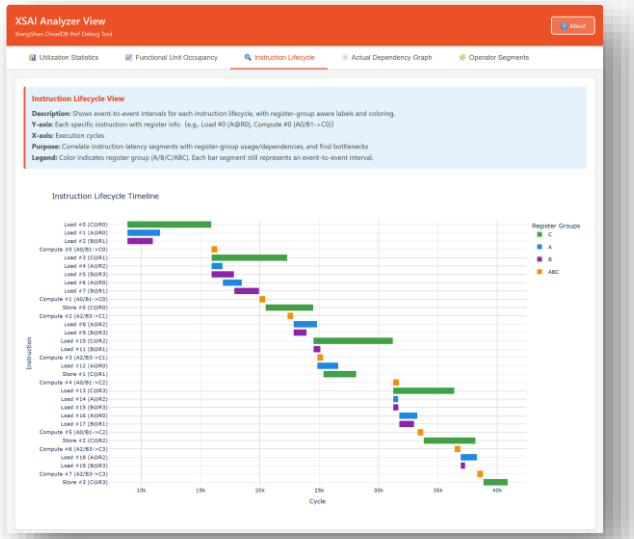
- XSAI's solution
  - Only reorder async instructions by software-ROB
  - Has been applied to **XSAI's development and CI**



Visualization Toolkit for Perf Analysis



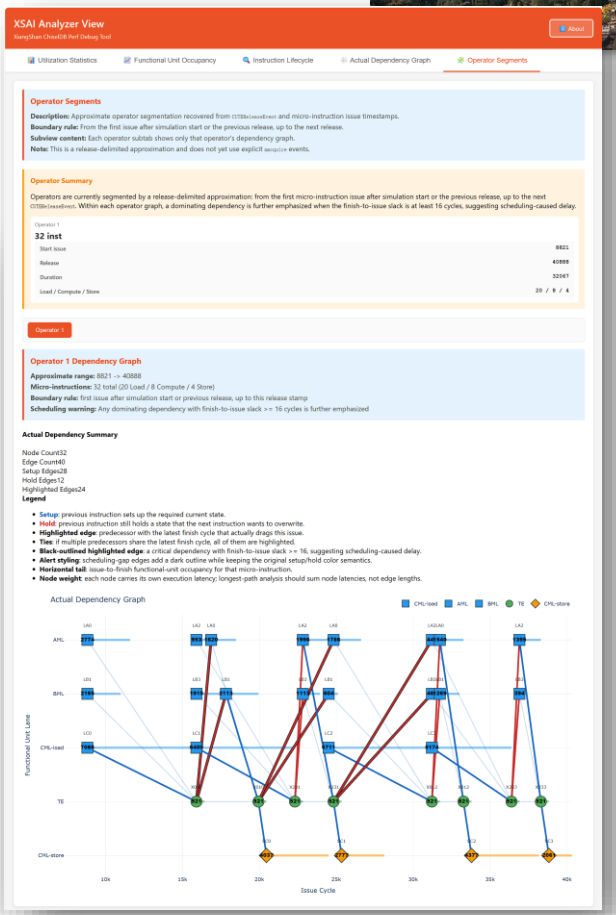
- RedLeaves
  - Based on XiangShan ChiselDB
  - Export an HTML report using scripts



Instruction Lifecycle



Dependency Graph

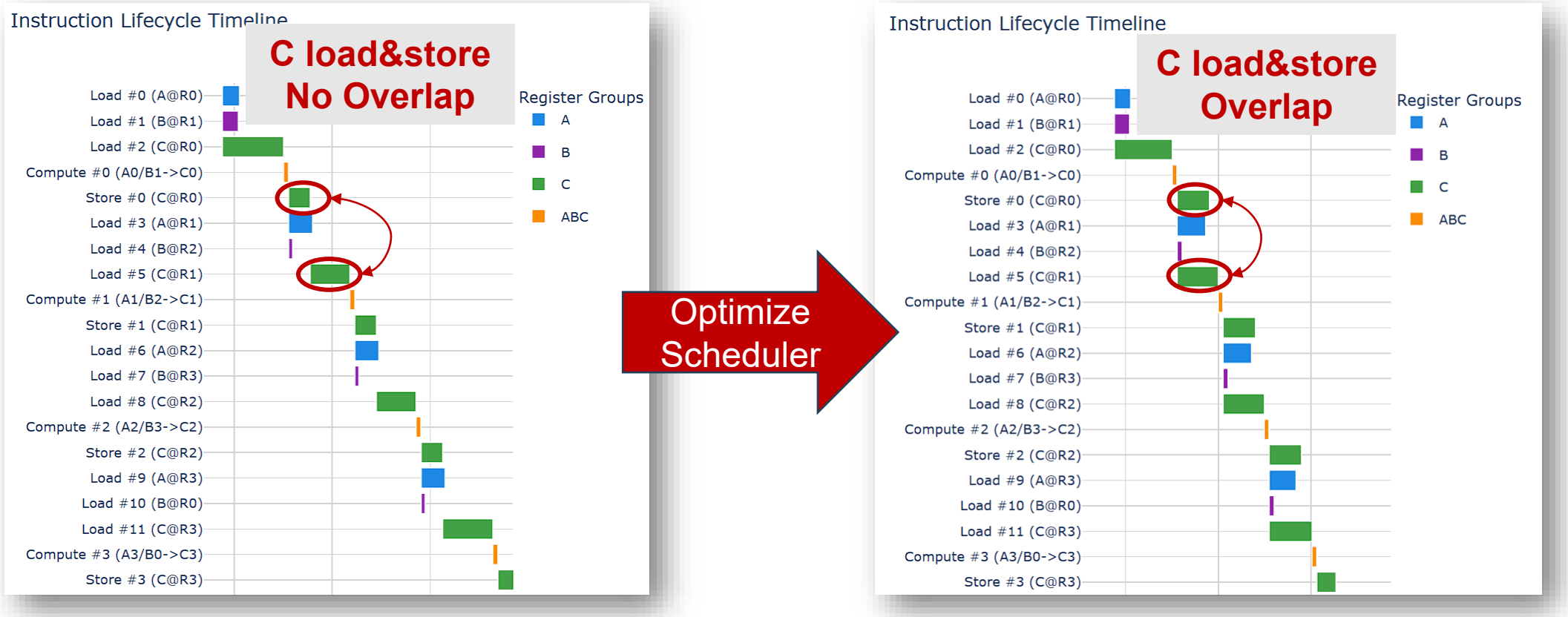


Operator Segments

An optimization case for Perf Analysis

- Split C load and store (CUTE #11)
  - C load and C store can overlap

10% speed up



Sampling for Evaluation

Full Workload

- High accuracy
- As LLM scale grows, cannot be completed on simulators / RTL

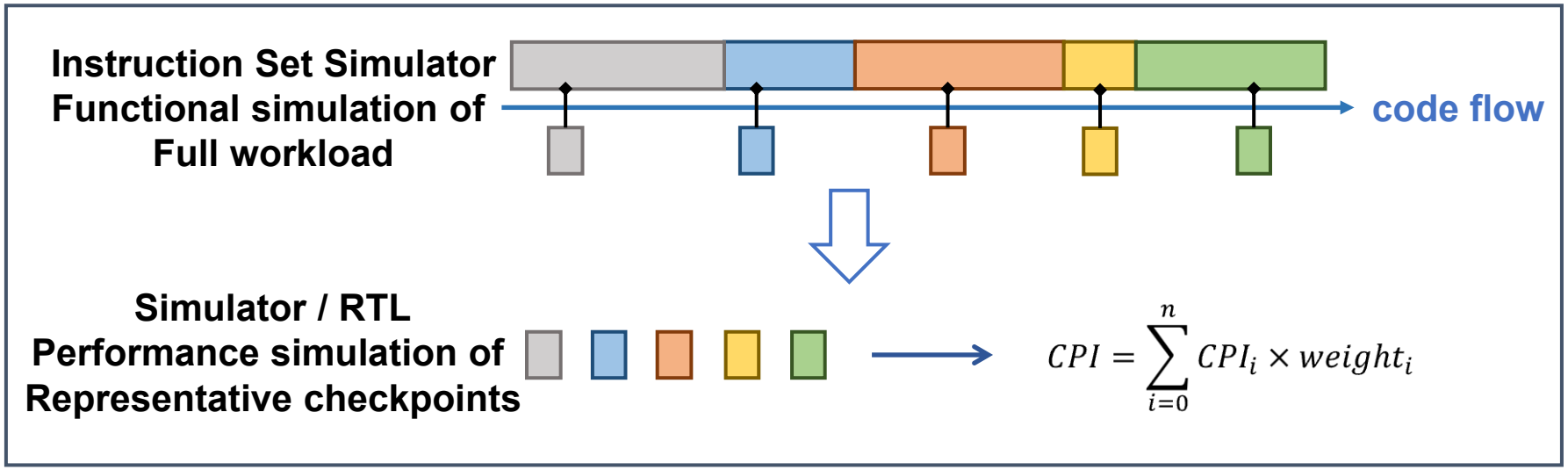
Operator Benchmark

- Small-scale, fast simulation
- Require manual extraction
- Cannot reflect system-level optimization benefits

Sampling Checkpoints

- Preserve realistic program characteristics
- Balance accuracy and simulation speed

- Sampling-based Evaluation Workflow:

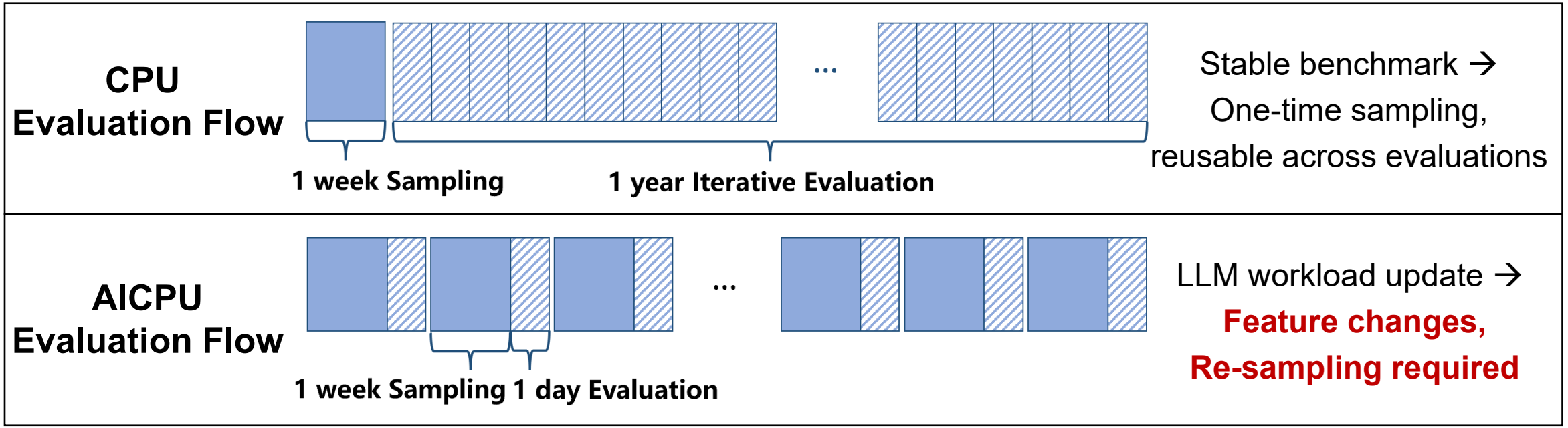


- **Single-core CPU: <2% error, >500× speedup**
- **Effective for multi-core CPU, GPU …**

New Challenge for Sampling

- Models evolve rapidly
- Framework optimizations diversify
- Accelerator ISAs evolve continuously

→ **LLM workloads update frequently**



Sampling Efficiency: The New Bottleneck in LLM Evaluation

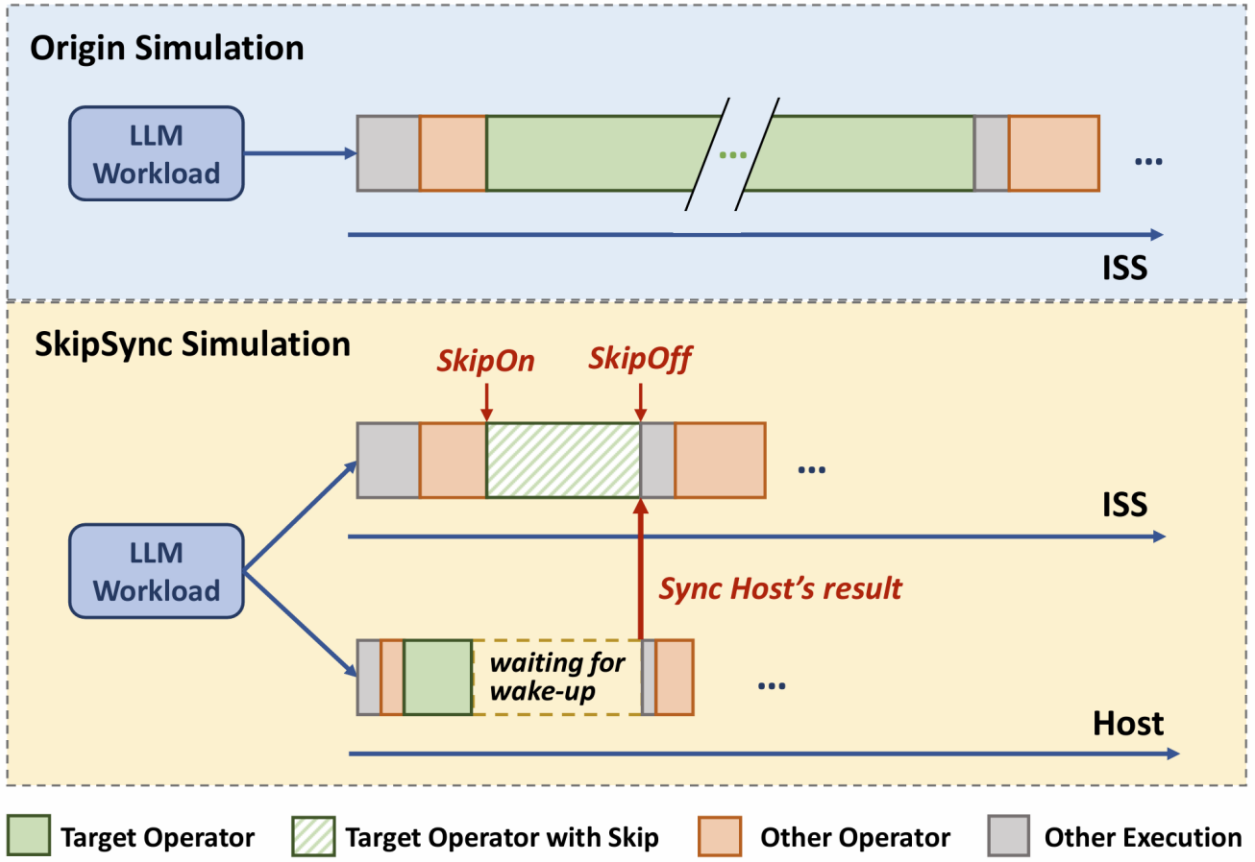
SkipSync: Accelerating Sampling of LLM Inference

➢ Skip

- Skips the execution phase of accelerator instructions within target operators (SkipOn - SkipOff)

➢ Sync

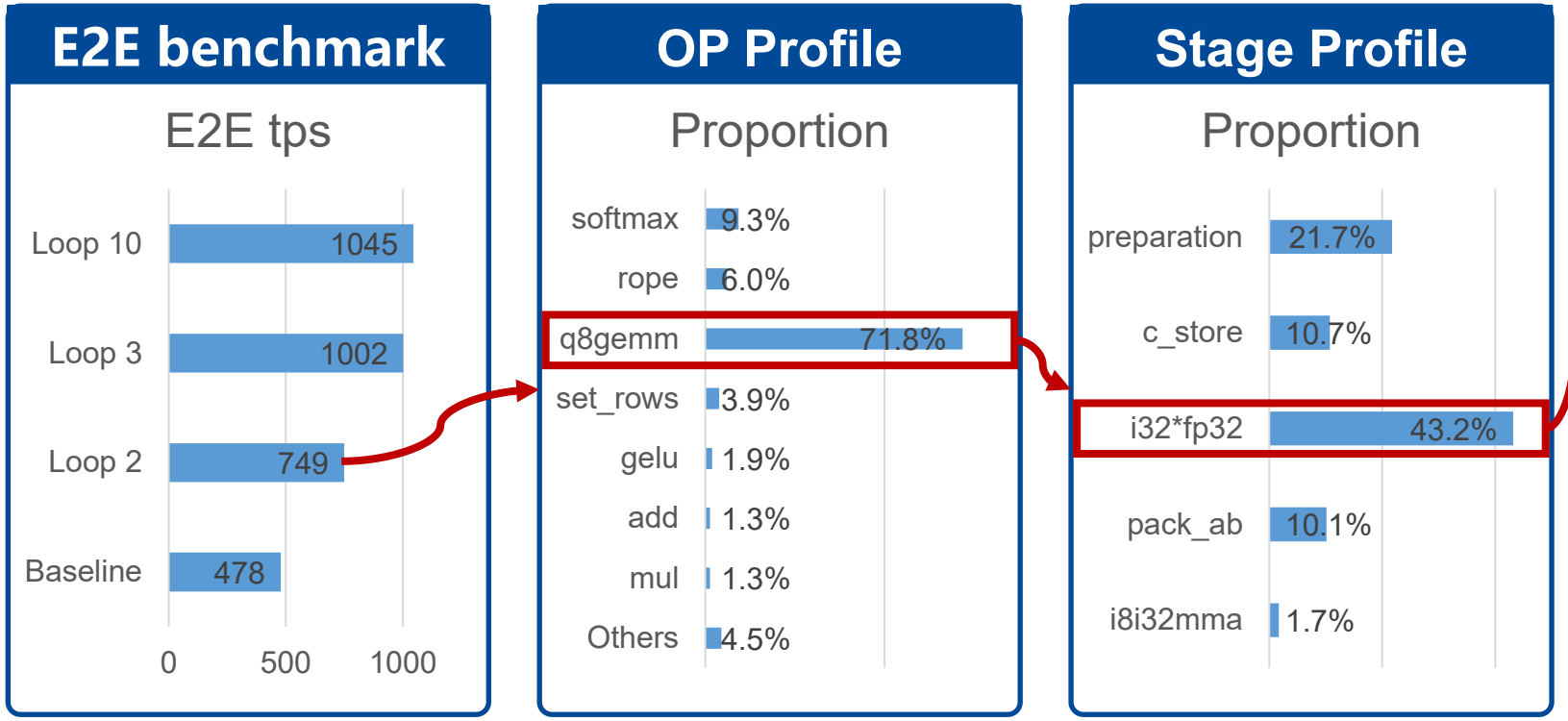
- Resolve memory state deviations caused by Skip
- Synchronize the host results back to the ISA simulator after operators



✓ **High extensibility for new accelerator instructions**
✓ **~5× speedup in XSAI LLM workload sampling, improving evaluation efficiency**

Kernel Design Agent (KDA)

- A top-down analysis agent flow
- **Open-source RTL** can enhance kernel optimization



Post-processing

- ~~Auto-vectorized~~

Optimize

- Explicit RVV intrinsics, considering
  - XSAI's vector
  - L1 DCache

1350M tokens
130 hours
For 10 iterations

FPGA Demonstration

- **Inference task on an FPGA**
  - Llama-2 15M model
  - The processor is trimmed
  - Scores don’t represent final perf



XSAI Roadmap

- XSAI V2R2A
  - Based on Kunminghu V2R2
  - Matrix: int8/bf16/fp16
  - Vector: V2R2 + bf16 (bf16 will be reused in KMH-V3)
  - HBL2: Support CHI protocol
  - Software: Support LLVM and Triton
  - Demo on **RISC-V Summit China 2026**
- XSAI V3R1A
  - Based on Kunminghu V3R1
  - Matrix: nvfp4/mxfp4/mxfp8/int8/bf16/fp16
  - Vector: V3R1 + fp8/fp4

Takeaways

- AI inference is becoming a system workload
  - Scheduling, KV cache, sampling, control flow, and memory movement are on the critical path
- CPU is moving from host processor to inference participant
  - Not replacing GPU/NPU, but cooperating with them in memory, control, and irregular compute
- XSAI = CPU-native programmability + AI-oriented throughput
  - XiangShan base, HBL2, async matrix, verification, perf tools, and kernel design flow

Together for a Shared Future

- Feel free to contact us through email or file issues on GitHub!
  - https://github.com/OpenXiangShan/XSAI

Github Repository

Biweekly Report